



Semestre 2 | Outils Numériques & Modélisation II
Cycle Préparatoire Intégré – ESIROI

Travaux Pratiques Résolution Numérique de l'Équation de la Chaleur 1D

*Simulation de la Diffusion Thermique
dans une Barre Métallique*

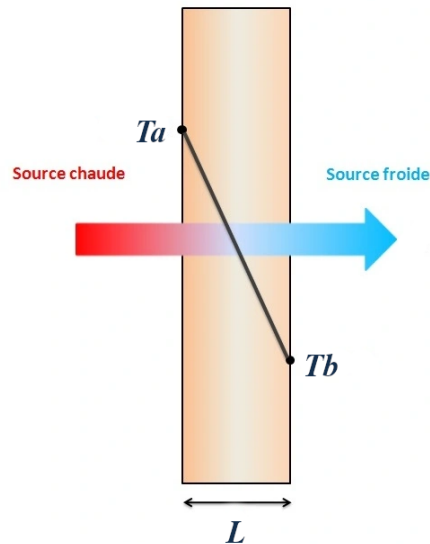
- 🕒 **Durée :** 4 heures
- 🔧 **Outils :** Python 3, NumPy, Matplotlib
- 📖 **Niveau :** CPI1 – Semestre 2
- 📌 **Thèmes :** EDP, Différences finies, Stabilité

0 Table des matières

1	Rappels Théoriques	2
1.1	Le Phénomène Physique	2
1.2	Discretisation par Différences Finies	3
1.3	Condition de Stabilité	3
2	Mise en Place du Code de Base	4
2.1	Paramètres de Simulation	4
2.2	Initialisation du Domaine	5
2.3	Boucle Temporelle Principale	5
2.4	Visualisation des Résultats	6
2.5	Encapsulation dans des fonctions	7
3	Optimisation avec NumPy ⚡	8
3.1	Version Vectorisée	8
3.2	Comparaison des Temps de Calcul	9
4	Analyse de la Stabilité Numérique	10
4.1	Influence du Paramètre r	10
4.2	Influence du Raffinement Spatial	11
5	Cas Physiques Plus Réalistes 🧪	12
5.1	Externaliser l'initialisation de la température	12
5.2	Condition Initiale en « Pic » de Température	12
5.3	Carte de Chaleur Spatio-Temporelle (Heatmap)	13
5.4	Condition adiabatique	14
6	Solution Analytique et Validation 📊	14
6.1	Solution Analytique Exacte	14
6.2	Implémentation et Comparaison	15
6.3	Calcul de l'Erreur	16

Partie 1 – Rappels Théoriques

1.1 Le Phénomène Physique



La conduction thermique dans un milieu solide est gouvernée par la **loi de Fourier**. Pour une barre homogène de longueur L , la température $T(x, t)$ évolue selon l'équation de la chaleur 1D :

Équation de la Chaleur 1D

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2} \quad (1)$$

où les grandeurs physiques sont :

Symbole	Signification	Unité
$T(x, t)$	Température au point x et à l'instant t	$^{\circ}\text{C}$ ou K
$\alpha = k/(\rho c_p)$	Diffusivité thermique	m^2/s
k	Conductivité thermique	$\text{W}/(\text{m}\cdot\text{K})$
ρ	Masse volumique	kg/m^3
c_p	Chaleur spécifique	$\text{J}/(\text{kg}\cdot\text{K})$

Remarque

Interprétation physique : Cette équation traduit que la variation temporelle de température en un point est proportionnelle à la *courbure spatiale* du profil de température. Un point “plus froid que ses voisins” ($\partial^2 T / \partial x^2 > 0$) va se réchauffer, et inversement.

Cette équation peut être obtenue en exprimant la variation de l'énergie interne d'un

volume de contrôle due aux flux thermique conductif aux frontières de ce volume.

1.2 Discrétisation par Différences Finies

Pour résoudre numériquement cette équation, on discrétise le domaine :

- ➔ **Dans l'espace** : on divise la barre $[0, L]$ en N intervalles de taille $\Delta x = L/N$, donnant $N + 1$ nœuds aux positions $x_i = i \cdot \Delta x$, avec $i = 0, 1, \dots, N$.
- ➔ **Dans le temps** : on avance par pas de temps Δt , aux instants $t^n = n \cdot \Delta t$, avec $n = 0, 1, 2, \dots$.

On note $T_i^n \approx T(x_i, t^n)$. Les dérivées sont approchées par :

$$\frac{\partial T}{\partial t} \approx \frac{T_i^{n+1} - T_i^n}{\Delta t} \quad (\text{différence finie décentrée } \textit{avant}) \quad (2)$$

$$\frac{\partial^2 T}{\partial x^2} \approx \frac{T_{i+1}^n - 2T_i^n + T_{i-1}^n}{\Delta x^2} \quad (\text{différence finie } \textit{centrée}) \quad (3)$$

le schéma temporel utilisé est donc un schéma Euler explicite. D'autres schémas (explicites ou implicites) sont possibles (RK4, AM2, ...).

En substituant dans l'équation de la chaleur, on obtient :

 **Schéma Euler explicite en temps, centré en espace**

$$T_i^{n+1} = T_i^n + r(T_{i+1}^n - 2T_i^n + T_{i-1}^n) \quad \text{avec} \quad r = \frac{\alpha \Delta t}{\Delta x^2} \quad (4)$$

Le paramètre r est appelé **nombre de Fourier discret** (ou paramètre de stabilité).

1.3 Condition de Stabilité

 **Attention – Condition de Stabilité**

Le schéma Euler explicite n'est **stable** que si :

$$r = \frac{\alpha \Delta t}{\Delta x^2} \leq \frac{1}{2} \quad (5)$$

Si cette condition n'est **pas** respectée, la solution numérique **diverge** : des oscillations croissantes non physiques apparaissent.

</> Partie 2 – Mise en Place du Code de Base

Contexte Physique

On considère une **barre de cuivre** de longueur $L = 1$ m, initialement à une température uniforme $T_0 = 0^\circ\text{C}$. Les deux extrémités sont maintenues à des températures constantes (**conditions de Dirichlet**) :

 $T(0, t) = 100^\circ\text{C}$ (extrémité gauche chauffée)

 $T(L, t) = 0^\circ\text{C}$ (extrémité droite froide)

Diffusivité thermique du cuivre : $\alpha = 1,11 \times 10^{-4} \text{ m}^2/\text{s}$

2.1 Paramètres de Simulation

Complétez et exécutez le code suivant pour définir les paramètres :

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.cm as cm
4
5 # --- Parametres physiques ---
6 L = 1.0 # Longueur de la barre (m)
7 alpha = 1.11e-4 # Diffusivite thermique du cuivre (m^2/s)
8 T_gauche = 100.0 # Temperature a x=0 (degC)
9 T_droite = 0.0 # Temperature a x=L (degC)
10 T_init = 0.0 # Temperature initiale uniforme (degC)
11
12 # --- Parametres numeriques ---
13 N = 20 # Nombre d'intervalles spatiaux
14 dx = L / N # Pas spatial (m)
15 r = 0.4 # Nombre de Fourier discret (choisir r <= 0.5)
16 dt = ... # Pas de temps : a calculer depuis r, dx et alpha
17 t_fin = 3000 # Duree de simulation (s)
18 nt = int(t_fin / dt)
19
20 print(f"dx = {dx:.4f} m")
21 print(f"dt = {dt:.4f} s")
22 print(f"r = {r:.4f} (doit etre <= 0.5)")
23 print(f"Nombre de pas de temps : {nt}")

```

Listing 1 – Initialisation des paramètres

☰ Travail Demandé (au minimum)

- ☑ Calculez Δt pour obtenir $r = 0,4$. Rappelez la formule utilisée.
- ☑ Vérifiez que la condition de stabilité est bien respectée.
- ☑ Combien y a-t-il de nœuds intérieurs dont la température est *inconnue* (i.e.

non imposée) ?

2.2 Initialisation du Domaine

```

1 # --- Creation de la grille spatiale ---
2 x = np.... # N+1 points de x=0 a x=L
3
4 # --- Condition initiale ---
5 T = np.ones(N+1) * T_init
6
7 # --- Application des conditions aux limites ---
8 T[0] = ...
9 T[-1] = ...
10
11 # --- Visualisation du profil initial ---
12 fig, ax = plt.subplots(figsize=(8, 4))
13 ax.plot(x, T, 'o-', color='royalblue', linewidth=2,
14         markersize=6)
15 ax.set_xlabel('Position x (m)')
16 ax.set_ylabel('Temperature T (degC)')
17 ax.set_title("Profil initial de temperature T(x, t=0)")
18 ax.grid(True, linestyle='--', alpha=0.5)
19 plt.tight_layout()
20 plt.show()

```

Listing 2 – Grille spatiale et condition initiale

2.3 Boucle Temporelle Principale

```

1 # --- Stockage periodique de la solution ---
2 # pourquoi ?
3 save_frac = 10
4 save_step = max(1, nt // save_frac)
5 T_save = [T.copy()]
6 t_save = [0.0]
7
8 T_new = T.copy()
9
10 for n in range(1, nt+1):
11     # Mise a jour des noeuds interieurs (i = 1 a N-1)
12     for i in range(1, N):
13         T_new[i] = ...
14
15     T = T_new.copy() # pourquoi "copy" ?
16
17     # Sauvegarde periodique
18     if n % save_step == 0:
19         T_save.append(T.copy()) # pourquoi "copy" ?
20         t_save.append(n * dt)
21

```

```

22 T_save = np.array(T_save)
23 print(f"Nombre de profils sauvegardés : {len(T_save)}")

```

Listing 3 – Schéma Euler explicite – boucle temporelle

☰ Travail Demandé (au minimum)

- ☰ Étudiez et complétez le code
- ☰ A quoi sert `T_new` ?
- ☰ Discuter de l'importance des commandes `.copy()`

Analyse d'un cas minimaliste

```

1  import numpy as np
2
3  T = np.array([1.0, 2.0, 3.0])
4  T_save = []
5
6  T_save.append(T)           # stocke la référence
7  print(T_save[0])          # [1. 2. 3.]
8
9  T[0] = 999                 # on modifie T
10 print(T_save[0])           # [999. 2. 3.] modifié aussi !

```

Avec `.copy()` :

```

1  T_save.append(T.copy())    # stocke une copie indépendante
2  T[0] = 999
3  print(T_save[0])          # [1. 2. 3.] inchangé

```

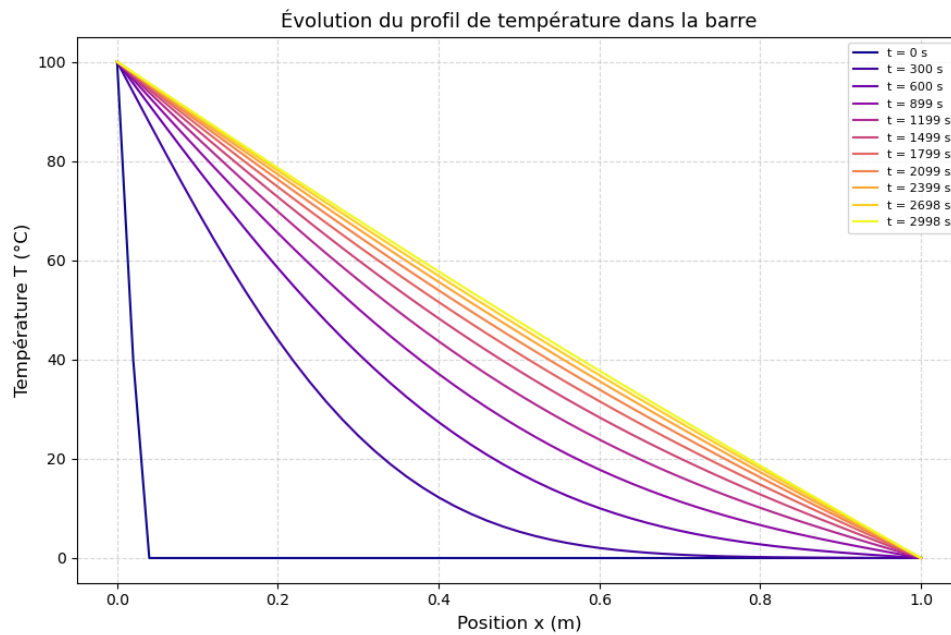
2.4 Visualisation des Résultats

```

1  fig, ax = plt.subplots(figsize=(9, 6))
2
3  colors = cm.plasma(np.linspace(0, 1, len(T_save)))
4  .
5  .
6  .

```

Listing 4 – Tracé des profils de température



☰ Travail Demandé (au minimum)

- ☑ Tracer les résultats tels que sur la figure ci-dessus.
- ☑ Décrivez qualitativement l'évolution du profil de température.
- ☑ Vers quel profil le système semble-t-il converger quand $t \rightarrow +\infty$? Justifiez physiquement.
- ☑ Quel est le profil théorique exact à l'état stationnaire? (*Indice* : résolvez $\frac{d^2T}{dx^2} = 0$ avec les conditions aux limites données.)

2.5 Encapsulation dans des fonctions

Regroupez le code précédent dans deux fonctions, une pour le tracé des profile de T et l'autre pour le calcul. Elles auront les signatures ci-dessous :

☰ Travail Demandé (au minimum)

- ☑ Réaliser la 1ere fonction
- ☑ la tester
- ☑ Réaliser la seconde fonction
- ☑ Tester l'ensemble

```

1 def tracer_T_x(x, T, t):
2     fig, ax = plt.subplots(figsize=(9, 6))
3     colors = cm.plasma(np.linspace(0, 1, len(T)))
4     .
5     .
6     plt.show()
```

Listing 5 – Mise en fonction


```

1 def simulation_chaleur(L, alpha, N, r, t_fin, T_gauche,
2   T_droite, T_init, save_frac=10):
3     """
4     Résout l'équation de la chaleur 1D par schéma Euler
5     explicite.
6
7     Paramètres :
8     -----
9     L          : longueur de la barre (m)
10    alpha       : diffusivité thermique (m²/s)
11    N           : nombre d'intervalles spatiaux
12    r           : nombre de Fourier discret (r <= 0.5 pour la
13                  stabilité)
14    t_fin       : durée totale de simulation (s)
15    T_gauche    : température à x=0 (°C)
16    T_droite    : température à x=L (°C)
17    T_init      : température initiale uniforme (°C)
18
19    Retourne :
20    -----
21    x, t_save, T_save
22    """
23
24    dx = L / N
25
26    .
27    .
28    .
29
30    return x, np.array(t_save), np.array(T_save)

```

Listing 6 – Mise en fonction

⚡ Partie 3 – Optimisation avec NumPy

La double boucle Python de la Partie 1 est **très lente** pour de grandes grilles. NumPy permet de **vectoriser** les calculs et d'éviter les boucles explicites.

3.1 Version Vectorisée

```

1 def simulation_chaleur_vect(L, alpha, N, r, t_fin, T_gauche,
2   T_droite, T_init, save_frac=10):
3     """
4     Résout l'équation de la chaleur 1D par schéma FTCS
5     (version vectorisée NumPy).
6
7     Paramètres
8     -----
9     L          : longueur de la barre (m)
10    alpha       : diffusivité thermique (m²/s)

```

```

10     N           : nombre d'intervalles spatiaux
11     r           : nombre de Fourier discret (r <= 0.5)
12     t_fin       : duree totale de simulation (s)
13     T_gauche    : temperature a x=0 (degC)
14     T_droite    : temperature a x=L (degC)
15     T_init      : temperature initiale uniforme (degC)
16     save_frac   : fraction de pas de temps a sauvegarder
17
18     Retourne
19     -----
20     x, t_save, T_save
21     """
22     dx = L / N
23     dt = r * dx**2 / alpha
24     nt = int(t_fin / dt)
25
26     x = np.linspace(0, L, N+1)
27     T = np.ones(N+1) * T_init
28     T[0] = T_gauche
29     T[-1] = T_droite
30
31     save_every   = max(1, nt // save_step_frac)
32     T_save, t_save = [T.copy()], [0.0]
33
34     for n in range(nt):
35         # Version vectorisee : une seule ligne !
36         T[1:-1] = T[1:-1] + r * (T[2:] - 2*T[1:-1] + T[:-2])
37         # T[0] et T[-1] ne sont pas modifiees (CL preservees)
38
39         if (n + 1) % save_every == 0:
40             T_save.append(T.copy())
41             t_save.append((n + 1) * dt)
42
43     return x, np.array(t_save), np.array(T_save)

```

Listing 7 – Fonction vectorisée de la simulation

☰ Travail Demandé (au minimum)

- ☰ Expliquez ligne par ligne ce que réalise l'instruction :
 $T[1:-1] = T[1:-1] + r * (T[2:] - 2*T[1:-1] + T[:-2])$
 En particulier, à quoi correspondent $T[2:]$, $T[1:-1]$ et $T[:-2]$?
- ☰ Appelez `simulation_chaleur_vect()` avec les mêmes paramètres qu'en Partie 1 et vérifiez que vous obtenez les mêmes résultats.

3.2 Comparaison des Temps de Calcul

```

1     import time
2
3     # --- Version avec boucle (Partie 1) ---

```

```

4      debut = time.time()
5      # ... Appel à la fonction non vectorisée ici ...
6      fin = time.time()
7      print(f"Temps boucle for : {fin - debut:.3f} s")
8
9      # --- Version vectorisee ---
10     debut = time.time()
11     # ... Appel à la fonction vectorisée ici ...
12     fin = time.time()
13     print(f"Temps vectorise : {fin - debut:.3f} s")

```

Listing 8 – Mesure des temps de calcul

☰ Travail Demandé (au minimum)

- ☑ Comparez les temps de calcul obtenus pour un **N assez grand** (ex. $N = 200$).
- ☑ Que concluez-vous sur l'intérêt de la vectorisation NumPy ?

⌕ Partie 4 – Analyse de la Stabilité Numérique

4.1 Influence du Paramètre r

Testez les trois valeurs du paramètre de Fourier résumées dans le tableau suivant :

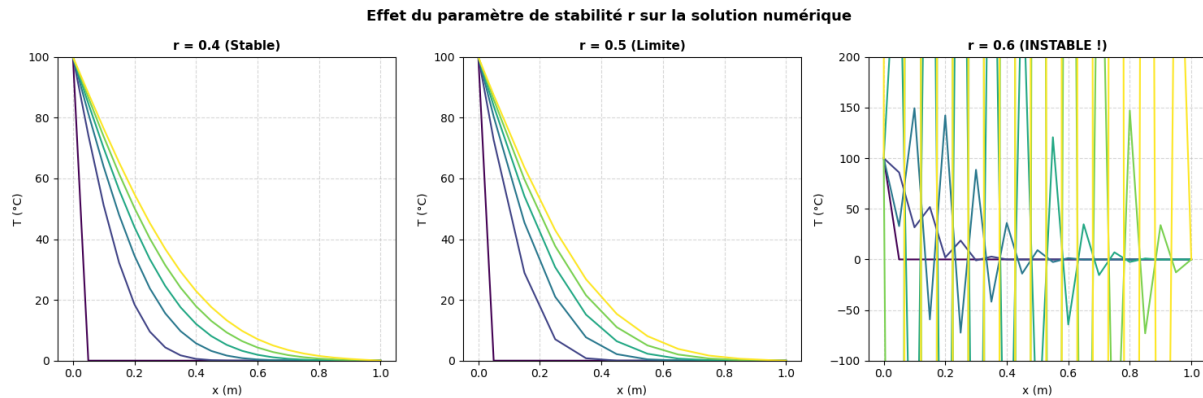
Cas	Valeur de r	Statut	Comportement attendu
A	$r = 0,4$	Stable	Solution physique correcte
B	$r = 0,5$	Limite	Stable mais précision réduite
C	$r = 0,6$	INSTABLE!	Oscillations croissantes – divergence

```

1  fig, axes = plt.subplots(1, 3, figsize=(15, 5))
2
3  r_values = [0.4, 0.5, 0.6]
4  titres   = ['r = 0.4  (Stable)', 'r = 0.5  (Limite)',
5             'r = 0.6  (INSTABLE !)']
6
7  for ax, r_val, titre in zip(axes, r_values, titres):
8      ... = simulation_chaleur_vect(... ,..., save_frac=5)
9      colors = cm.viridis(np.linspace(0, 1, len(T_sv_r)))
10     .
11     .
12     .
13     plt.tight_layout()
14     plt.show()

```

Listing 9 – Comparaison de trois valeurs de r



Instabilité Numérique

Pour $r > 0,5$, des oscillations **non physiques** apparaissent et s'amplifient au cours du temps. La solution numérique est alors totalement dénuée de sens physique. Il est **impératif** de vérifier la condition de stabilité avant tout calcul.



Travail Demandé (au minimum)

- ☞ complétez le code de manière à obtenir la figure ci-dessus.
- ☞ Décrivez ce que vous observez pour chaque valeur de r .
- ☞ Que se passe-t-il physiquement pour $r > 0,5$? Est-ce acceptable?
- ☞ Comment choisiriez-vous Δt en pratique pour garantir la stabilité tout en conservant un temps de calcul raisonnable?

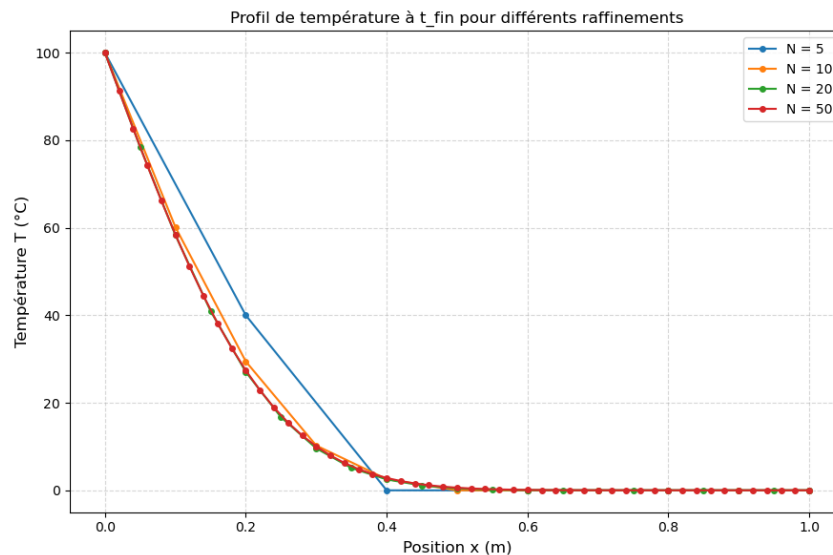
4.2 Influence du Raffinement Spatial

Comparez les solutions pour différentes valeurs de N (en gardant $r = 0,4$ fixé) :

```

1      N_values = [5, 10, 20, 50]
2
3      fig, ax = plt.subplots(figsize=(9, 6))
4      .
5      .
6      .
7      plt.show()
```

Listing 10 – Convergence par raffinement spatial



☰ Travail Demandé (au minimum)

- ☑ complétez le code de manière à obtenir la figure ci-dessus.
- ☑ Les résultats convergent-ils quand N augmente ?

🔗 Partie 5 – Cas Physiques Plus Réalistes 🧪

5.1 Externaliser l'initialisation de la température

Réaliser une nouvelle version de `simulation_chaleur_vect` qui prend comme paramètre une fonction dont le rôle sera d'initialiser la température.

```
1 def simulation_chaleur_vect(L, alpha, N, r, t_fin, T_gauche,
2   T_droite, f_init, save_step=10):
3   .
```

Listing 11 – Fonction vectorisée de la simulation

5.2 Condition Initiale en « Pic » de Température

Simulez le cas d'une **zone chaude localisée au centre** de la barre, avec les deux extrémités maintenues à 0°C . La condition initiale est une **gaussienne** :

$$T(x, 0) = T_{\max} \exp\left(-\frac{(x - L/2)^2}{2\sigma^2}\right) \quad (6)$$

avec $T_{\max} = 100^\circ\text{C}$ et $\sigma = 0,05\text{ m}$.

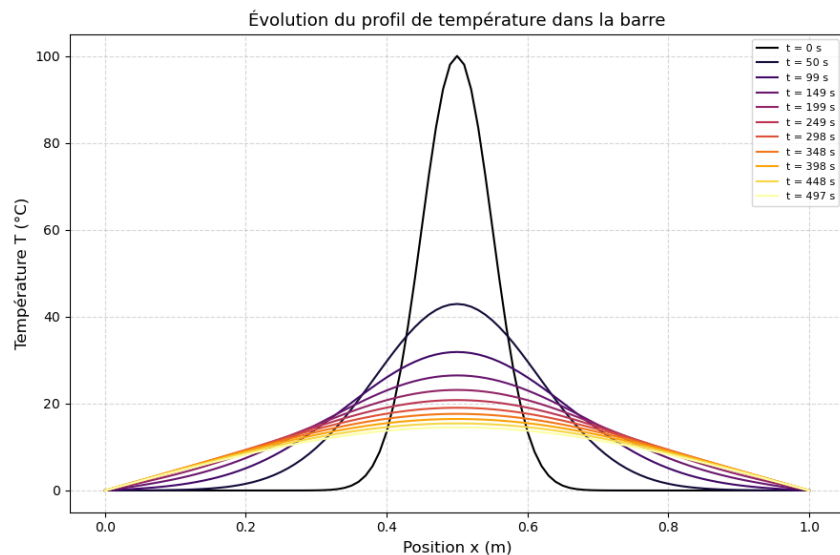
```
1 def f_init_gauss(x):
2   sigma = 0.05
3   Tmax = 100
4   L = x[-1]
```

```

5     return ...
6
7 x, t_save, T_save = simulation_chaleur_vect(...
8 .
9 .

```

Listing 12 – Diffusion d'un pic gaussien



☰ Travail Demandé (au minimum)

- ☰ complétez le code de manière à obtenir la figure ci-dessus.
- ☰ Décrivez l'évolution du pic de température.
- ☰ Vers quoi converge la température dans toute la barre quand $t \rightarrow +\infty$? Pourquoi?
- ☰ Refaites la simulation avec $\sigma = 0,02$ m. Comment la diffusion est-elle influencée par la largeur initiale du pic?

5.3 Carte de Chaleur Spatio-Temporelle (Heatmap)

```

1 def f_init_zero(x):
2     return np.zeros_like(x)
3
4 x, t_map, T_map = simulation_chaleur_vect(L, alpha, N=100,
5     r=0.4, t_fin=3000, T_gauche=100, T_droite=0,
6     f_init=f_init_zero, save_frac=1)
7
8 # --- Heatmap ---
9 fig, ax = plt.subplots(figsize=(10, 6))
10 c = ax.contourf(x, t_map, T_map, levels=50, cmap='hot')
11 fig.colorbar(c, ax=ax, label='Température (°C)')
12 ax.set_xlabel('Position x (m)', fontsize=12)

```

```

13 ax.set_ylabel('Temps t (s)', fontsize=12)
14 ax.set_title("Carte spatio-temporelle de la température
    T(x,t)", fontsize=13)
15 plt.tight_layout()
16 plt.show()

```

Listing 13 – Représentation en heatmap de $T(x, t)$

☰ Travail Demandé (au minimum)

- ☑ Interprétez la carte obtenue.
- ☑ Identifiez les zones où la diffusion est la plus rapide.
- ☑ Qu'observe-t-on à grande valeur de t ? Commentez.
- ☑ Appliquer au cas du profil en pic de température.

5.4 Condition adiabatique

Dans le cas où la bord droit de la barre serait calorifugée (isolé thermiquement), nous aurons à cette endroit une condition aux limites de type flux thermique nul :

🏠 C.L. adiabatique

$$\left. \frac{\partial T}{\partial x} \right|_{x=L} = \frac{T_N - T_{N-1}}{\Delta x} = 0 \quad (7)$$

$$\implies T_N = T_{N-1}$$

☰ Travail Demandé (au minimum)

- ☑ Implémenter cette condition aux limites dans une nouvelle fonction.
- ☑ Tester et tracer les résultats. Commenter.

🔗 Partie 6 – Solution Analytique et Validation

6.1 Solution Analytique Exacte

Pour notre problème ($T(0, t) = 100$, $T(L, t) = 0$, $T(x, 0) = 0$), la solution exacte est :

🏠 Solution Analytique (série de Fourier)

$$T(x, t) = T_g + (T_d - T_g) \frac{x}{L} + \sum_{n=1}^{+\infty} C_n \sin\left(\frac{n\pi x}{L}\right) \exp\left(-\alpha \left(\frac{n\pi}{L}\right)^2 t\right) \quad (8)$$

avec :

$$C_n = \frac{2}{n\pi} [(T_{init} - T_g) - (-1)^n (T_{init} - T_d)]$$

 Remarque

En pratique, la série converge rapidement. On tronque à $n = 50$ termes, ce qui donne une excellente précision pour $t > 0$.

6.2 Implémentation et Comparaison

```

1 def solution_analytique(x, t, L, alpha, T_gauche, T_droite,
2   T_init, N_termes=50):
3     """
4     Solution analytique de l'équation de la chaleur 1D.
5
6     Paramètres :
7     -----
8     x          : tableau des positions (array)
9     t          : instant (scalaire)
10    N_termes   : nombre de termes de la série de Fourier
11    """
12    # Partie stationnaire
13    T_st = T_gauche + (T_droite - T_gauche) * x / L
14
15    # Partie transitoire (série de Fourier)
16    .
17    .

```

Listing 14 – Solution analytique

```

1 # --- Comparaison numérique vs analytique ---
2 x_num, t_sv, T_sv_num = simulation_chaleur_vect(L=1.0,
3   alpha=1.11e-4, N=20, r=0.4, t_fin=3000, T_gauche=100,
4   T_droite=0, f_init=f_init_zero, save_frac=100)
5
6 # instants choisis
7 n_sv = len(t_sv)
8 n_compare = [int(n_sv/12), int(n_sv/4), int(n_sv/2), n_sv-1]
9
10 x_ana = np.linspace(0, 1.0, 200)
11
12 fig, axes = plt.subplots(2, 2, figsize=(12, 8))
13 axes = axes.flatten() # Transforme le tableau 'axes' 2D en
14   tableau 1D :
15
16 for ax, n in zip(axes, n_compare):
17     # Solution analytique
18     T_ana = solution_analytique(x_ana, ...
19     .
20     .
21
22 plt.suptitle("Comparaison Solution Numerique vs
23   Analytique", fontsize=13, fontweight='bold')

```



```

21 plt.tight_layout()
22 plt.show()

```

Listing 15 – Comparaison numérique/analytique

6.3 Calcul de l'Erreur

```

1 fig, ax = plt.subplots(figsize=(8, 5))
2
3 erreurs = []
4 for idx, t_val in enumerate(t_sv[1:]):
5     T_ana_pts = solution_analytique(x_num, t_val, L=1.0,
6                                     alpha=1.11e-4, T_gauche=100, T_droite= 0, T_init=0)
7
8     erreur = np.max(np.abs(T_sv_num[idx+1] - T_ana_pts))
9     erreurs.append(erreur)
10
11 ax.semilogy(t_sv[1:], erreurs, 'b-o', markersize=3)
12 ax.set_xlabel('Temps t (s)', fontsize=12)
13 ax.set_ylabel('Erreur max |T_num - T_ana| (řC)', fontsize=11)
14 ax.set_title("Évolution de l'erreur numérique au cours du temps",
15              fontsize=12)
16 ax.grid(True, which='both', linestyle='--', alpha=0.5)
17 plt.tight_layout()
18 plt.show()

```

Listing 16 – Évolution de l'erreur numérique

☰ Travail Demandé (au minimum)

- ☑ Les solutions numérique et analytique sont-elles en accord ?
- ☑ Comment évolue l'erreur au cours du temps ? Proposez une explication physique.
- ☑ Testez avec $N = 10$ puis $N = 100$. Quel est l'effet du raffinement sur l'erreur maximale ?